Storing Images and BLOB files in SQL Server

By Don Schlichting

Introduction

This article explores the data types and methods used for storing BLOBs (Binary Large Objects), such as images and sounds, inside SQL Server. Legacy data types used in SQL Server versions 2000 and SQL 2005 will be examined as well as the new SQL 2008 FILESTREAM binary type.

What are BLOBs

To start, we'll compare two types of files, ASCII and Binary. Most of the values stored in SQL Server consist of ASCII (American Standard Code for Information Interchange) characters. An overly simplified explanation of ASCII characters would be letters, numbers, and symbols found on the keyboard. A file containing only ASCII characters can be modified by a text editor such as Notepad without consequence. Binary files however, contain both ASCII characters and special control characters and byte combinations not found on the keyboard. An MP3 music file would be binary. Opening an MP3 inside Notepad and removing characters in an attempt to make the song shorter would result in the file being corrupted and not playable because Notepad is limited to ASCII characters and cannot correctly interpret or create binary bits. Other examples of binary data include images and EXE compiled programs. BLOBs then, are binary files that are large, or Binary Large Objects (BLOB).

Why store BOLBs in SQL Server?

There are justified reasons both for and against storing binary objects inside SQL server. We'll look at both sides. As a real world example, we'll consider a typical sales organization. There are usually product lines, or families of products being sold. A level below the product line would be the individual or discreet parts, we'll call them widgets. Each widgets has the standard inventory columns such as price, cost, quantity on hand, vendor, etc. In addition, many may have sales literature or brochures describing the widget. Often these brochures are electronic such as PDF, Power Point, or some type of image. One way of dealing with these electronic documents would be just to throw them up on a file server and create a directory for each widget. This will work, until customers or employees want an application they enter search parameters into and receive back the sales brochures that match. For example, "show me all documents for blue widgets that sell for less than $100". At this point, a database tied to an application will usually be involved. Therefore, for this series of articles, we'll create a Visual Studio application that connects to SQL Server to retrieve widget sales brochures.

File Storage Locations

One of the first questions is where to store the electronic brochures. Either the application could store the file system path information leading to the document, such as "d:\sales doc\widgeta-picture.jpg", inside a varchar column, leaving the actual document on the file system, or we could place the actual jpg file inside a binary or image column. A few key questions will help determine the best option.

- Performance: Are these binary objects performance hungry, such as a streaming video? If so, the file system may perform better than trying to stream the binary out of SQL Server.
- Size: Is the binary object to be retrieved large? Large being over 1 MB in size. If the object is large, the file system will typically be more efficient at presenting, or reading the object than SQL Server. If the binaries are small, say little images of each widget, then storing them inside SQL server will be more than adequate.
- Security: Is access to the binaries a high security concern? If the objects are stored in SQL Server, then security can be managed through the usual database access methods. If the files are stored on the file system, than alternative security methods will need to be in place.
- Client Access: How will the client access the database, ODBC, Native SQL Driver? For large streaming video, a client such as ODBC may time out or fail.
- Fragmentation: If the binaries will be frequently modified and are large, the file system may handle fragmentation better than SQL Server.
- Transactions: Do you need transactional control? If so, then SQL has a built in solution.

For an in-depth discussion on database vs. file system storage for Blobs, as well as where the previous 1MB size reference came from, see the Microsoft article: To BLOB or Not to BLOB, located at http://research.microsoft.com/research/pubs/view.aspx?msr_tr_id=MSR-TR-2006-45 .

Data Types

For this first example, we'll create an application that will store images of each product. Because these files are small, we'll opt to store them in SQL Server. In SQL 2000, there were two different families of data type options for these type of files, binary, and image. The Binary family includes three different data types. The Binary data type itself, which requires a fixed size. For this first example, because our images vary in size, we'll use the varbinary data type, the "var" standing for variable. The varbinary data type has a maximum length of 8,000 bytes. Starting in SQL 2005, "varbinary(max)" was included in the binary data type family. The keyword MAX indicates the size is unlimited. If the SQL version is before 2005 and the file size is greater than 8,000, then the Image data type can be used. It's a variable size type with a maximum file size of 2GB. Although the Image data type is included in SQL Server, it shouldn't be used. Microsoft says it's there for backwards compatibly and will be dropped at some point in the future. Therefore, this example will use the Binary type, the three versions of which are recapped below:

Binary: Fixed size up to 8,000 bytes.

VarBinary(n): Variable size up to 8,000 bytes (n specifies the max size).

VarBianry(max): Variable size, no maximum limit.

Conclusion

In the next article, we'll continue with BLOBs by creating a Visual Studio application that reads and writes to a SQL Server binary data type. The mechanics of the data type VarBinary(MAX) will be examined followed by the new SQL Server FILESTREAM option.